DevOps-Based Deployment and Orchestration

of a Multi-Agent Insurance System

Course Name: DevOps

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Mustafa shaikh	EN22CS301624
02	Naman Mehta	EN22CS301629
03	Naman Raghuvanshi	EN22CS301632
04	Nandini Sharma	EN22CS301635
05	Nishkarsh Sharma	EN22CS301659

Group Name: DO-20

Project Number: 09D6

Industry Mentor Name: Vaibhav Sir

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-2026

1. Introduction

The Multi-Agent Insurance Recommendation System is a web-based application designed to assist users in selecting suitable insurance policies based on their personal and financial requirements. The system leverages a multi-agent architecture, where different agents collaborate to validate user input, analyze policy data, generate recommendations, and provide comparative insights.

The application is developed using a lightweight technology stack with a Python Flask backend and a simple web-based frontend. In addition, DevOps practices such as containerization and continuous integration/continuous deployment (CI/CD) are incorporated to ensure efficient development, deployment, and scalability of the system.

1.1 Scope of the Document

This document provides a high-level design (HLD) of the Multi-Agent Insurance Recommendation System. It outlines the overall system architecture, major components, data flow, and deployment strategy.

The scope includes:
• Description of system architecture and design
• Overview of key components such as agents, backend, and frontend
• Explanation of data flow and interaction between modules
• High-level deployment and DevOps workflow
• Identification of key technologies used in the system

1.2 Intended Audience

- **Developers** – To understand the detailed system deployment and configuration.
- **DevOps Engineers** – To manage containerization, deployment, and orchestration using Docker and Kubernetes.
- **System Administrators** – To maintain and monitor the deployed application environment.
- **Project Evaluators / Reviewers** – To review the technical design and implementation of the system.
- **Students / Team Members** – To understand the deployment workflow and system structure.

1.3 System Overview

The Multi-Agent Insurance System is designed to handle insurance-related queries through a collaborative agent-based architecture. The system consists of multiple agents that work

together to process user requests and generate appropriate responses. Each agent is responsible for a specific task within the workflow.

The system processes user queries, retrieves relevant information, and generates structured outputs through coordinated interaction between the agents. The architecture follows a modular design that allows different components of the system to operate independently while collaborating to complete the overall task.

This design improves system organization, maintainability, and the ability to extend the system with additional functionalities in the future.

2. System Design

The Multi-Agent Insurance Recommendation System is a web-based application that provides personalized insurance policy suggestions based on user inputs. The system follows a modular architecture where different agents collaborate to process user requests and generate meaningful outputs.

The user interacts with the system through a web interface by providing details such as age, income, insurance type, and risk preference. The backend, developed using Python Flask, processes the request and coordinates with multiple agents to perform specific tasks.

The **Input Agent** validates the user data, the **Recommendation Agent** analyzes available policies and selects the most suitable options, the **Comparison Agent** generates a structured comparison of policies, and the **Query Agent** handles user queries through a rule-based chat mechanism.

The system uses a JSON-based data store to manage policy information. Additionally, DevOps practices such as version control, containerization, and CI/CD pipelines are integrated to ensure efficient development, deployment, and scalability of the application.

Overall, the system provides a simplified and automated approach for users to explore, compare, and understand insurance policies.

2.1 Application Design

The application follows a modular and layered architecture to ensure separation of concerns, scalability, and ease of maintenance. It is divided into frontend, backend, agent-based processing, and data layers.

The **frontend layer** is built using HTML, CSS, and JavaScript, providing an interface for users to input their details and view insurance recommendations and comparisons.

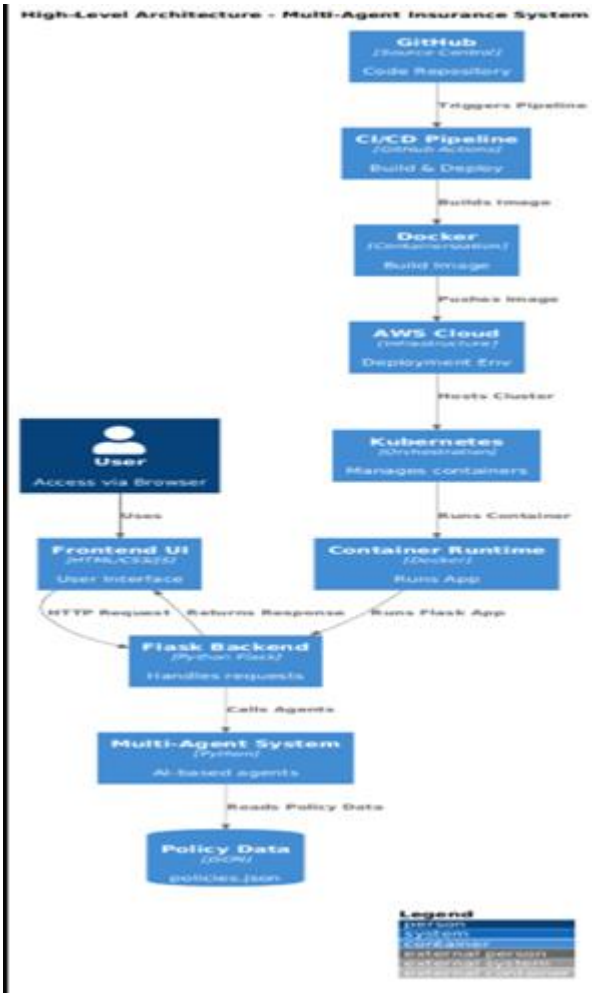
The **backend layer**, developed using Python Flask, acts as the central controller of the system. It handles incoming user requests, manages application logic, and coordinates with different agents to process data and generate responses.

The **multi-agent system** forms the core of the application logic. Each agent is responsible for a specific task:

- The Input Agent validates user inputs.
- The Recommendation Agent analyzes policy data and selects suitable options.
- The Comparison Agent generates a comparison of policies.
- The Query Agent handles user queries through predefined responses.

The **data layer** consists of a JSON file that stores insurance policy information. This data is accessed and processed by the agents to generate recommendations.

The application is designed to be lightweight and extensible, allowing easy integration of additional agents, data sources, or features in the future.



Main components include:

- **Source Code Repository** – Stores the application code and configuration files.
- **Container Configuration** – Defines how the application is packaged into a container image.
- **Container Image** – A packaged version of the application with all required dependencies.

- **Deployment Configuration** – Defines how the container runs in the cluster environment.
- **Service Layer** – Exposes the application so users can access it.

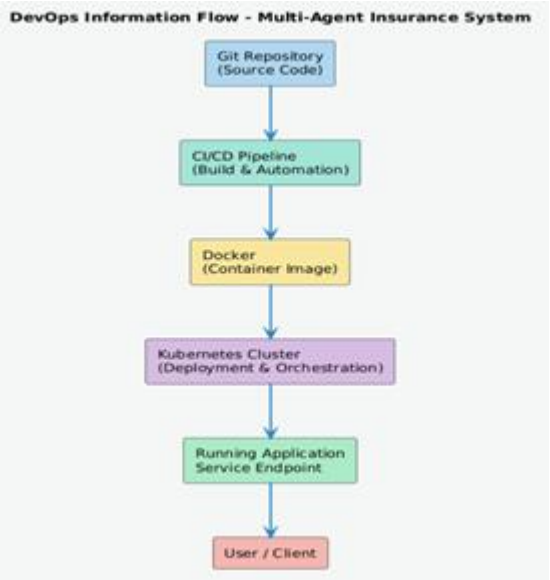
This design ensures that the application can be deployed consistently, managed efficiently, and updated without affecting system availability.

1.2. Process Flow

The process flow describes the sequence of steps involved in building and deploying the application within the system environment.

Process Flow Steps:

- **Code Management** – The application source code is stored and maintained in the repository.
- **Build Process** – The application is packaged into a container image using the configuration file.
- **Image Creation** – The container image is created with all required dependencies.
- **Deployment** – The container image is deployed into the cluster environment using deployment configuration files.
- **Service Exposure** – The application service is exposed so that it can be accessed by users.
- **Application Access** – Users interact with the deployed application through the provided service endpoint.



1.3 Information Flow

Information flows from the **source code repository (Git)** through the **automation and deployment pipeline**, where container images are built and deployed to the target infrastructure. The deployment configurations define how the application runs inside the container environment.

The deployment system manages the container lifecycle and ensures that the application is properly created, updated, and maintained in the runtime environment.

1.4 Components Design

• **Git Repository** – Stores the application source code and configuration files.
• **Automation / CI Pipeline** – Triggers build and deployment processes when code changes occur.
• **Docker** – Packages the application and its dependencies into container images.
• **Kubernetes** – Manages container deployment, scaling, and service exposure.

1.5 Key Design Considerations

• **Modularity** – System components are organized into separate configurations for easier maintenance.
• **Scalability** – The container orchestration platform allows the application to scale according to workload.
• **Reliability** – Deployment configurations ensure consistent application execution across environments.

1.6 API Catalogue

• **Kubernetes API** – Used to manage deployments, services, and container lifecycle operations within the cluster.

1.7 Use Case Overview

The Use Case Diagram represents the interaction between users, the system, and the DevOps engineer in the Multi-Agent Insurance Recommendation System.

Actors

• **User** – Interacts with the system to get insurance recommendations and ask queries.
• **Admin / DevOps Engineer** – Manages code, CI/CD pipeline, and deployment.

3.2 Use Cases

User

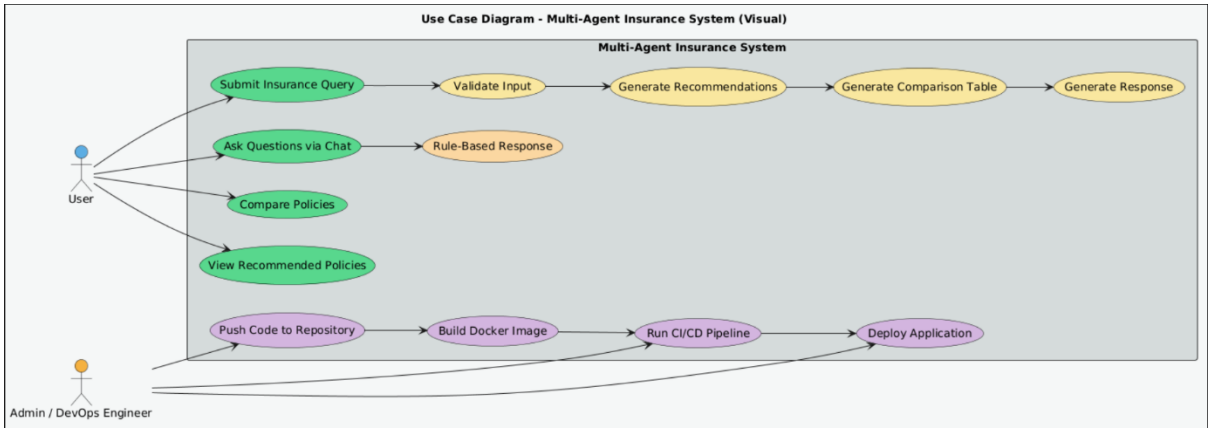
• Submit Insurance Query
• View Recommended Policies
• Compare Policies
• Ask Questions via Chat

System

• Validate Input
• Generate Recommendations
• Generate Comparison
• Generate Response
• Rule-Based Chat Response

DevOps

• Push Code to Repository
• Build Docker Image
• Run CI/CD Pipeline
• Deploy Application



2. Data Design

Data design describes how the system manages and organizes the data used during application execution and deployment. It defines how configuration files, container images, and deployment resources are structured and stored so that the system can operate efficiently.

2.1 Data Model

The data model defines the structure of the information used within the system components.

Main data elements include:

• **Source Code Files** – Contains the application logic and configuration files stored in the repository.
• **Container Configuration** – Defines how the application is packaged into a container image.
• **Container Image** – A packaged version of the application along with its dependencies.
• **Deployment Configuration Files** – YAML files that define how containers are deployed and managed.
• **Service Configuration** – Defines how the application is exposed for external access.

2.2 Data Migration

Data migration in the system mainly involves updating or modifying the knowledge sources used by the agents. When new insurance policy information becomes available, the knowledge base can be updated so that the Research Agent can retrieve the latest information.

The modular structure of the system allows new data sources or updated policy information to be integrated without affecting the overall system workflow.

3. Interfaces

- **User Interface**: Jenkins Web Dashboard for monitoring pipeline status.
- **Command Line Interface (CLI)**: Terraform and Git CLI for developers.

4. State and Session Management

State management is handled through the **Terraform State File** (.tfstate). This file acts as a database that records the current state of the infrastructure, ensuring that Terraform knows which resources to create, update, or delete during a run.

5. Caching

- **Docker Layers**: Jenkins uses Docker layer caching to speed up the building of telecom service images.
- **Terraform Plugins**: Provider plugins are cached locally to reduce deployment time.

6. Functional Requirements

The functional requirements describe the DevOps capabilities used to build, deploy, and manage the Multi-Agent Insurance Recommendation System.

6.1 Source Code Management

• **Version Control** – The project source code is maintained in a Git repository to track changes and manage collaboration.

• **Branch Management** – Developers can create branches to develop features and merge them into the main branch after testing.
• **Code Tracking** – Git maintains a history of changes for debugging and development purposes.

6.2 Application Containerization

• **Dockerfile Creation** – The application is packaged using a Dockerfile to create a portable container image.
• **Environment Consistency** – The Docker container ensures that the application runs consistently across development and deployment environments.
• **Dependency Management** – All required Python dependencies from the `requirements.txt` file are installed within the container.

6.3 Continuous Integration (CI)

• **Automated Build Process** – Each code update triggers an automated build process.
• **Dependency Installation** – The CI pipeline installs required dependencies before running the application.
• **Application Testing** – Basic checks ensure that the Flask application runs successfully.

6.4 Continuous Deployment (CD)

• **Automated Deployment** – After successful builds, the application container can be deployed automatically.
• **Container Execution** – The system runs the containerized Flask application on the target environment.
• **Service Availability** – The deployed application becomes accessible through a web interface.

6.5 Monitoring and Management

• **Application Logs** – System logs help monitor application activity and detect errors.
• **Deployment Monitoring** – Developers can track the status of builds and deployments through pipeline tools.
• **System Updates** – Updated application versions can be redeployed without manual configuration.

7. Non-Functional Requirements

These requirements describe the performance, reliability, and maintainability aspects of the DevOps pipeline.

7.1 Security Aspects

• **Secure Dependency Installation** – Dependencies are installed using verified package sources.
• **Access Control** – Only authorized users can modify the source code repository.
• **Configuration Security** – Sensitive environment variables can be managed securely during deployment.

7.2 Performance Aspects

• **Fast Build and Deployment** – Automation reduces the time required to deploy application updates.
• **Efficient Resource Usage** – Containerized applications use system resources efficiently.
• **Scalable Deployment** – The system can support scaling by deploying multiple container instances.

7.3 Reliability and Maintainability

• **Reproducible Environment** – Docker containers ensure consistent environments across development and production.
• **Automated Workflow** – CI/CD pipelines automate the build and deployment process.
• **Easy Maintenance** – Updates and fixes can be deployed quickly using automated pipelines.

8. References

Problem Statement: **Deploy Insurance App to Kubernetes**.

Project Group: **D6 - Group 09**.

Flask Documentation
https://flask.palletsprojects.com/

• Docker Documentation
https://docs.docker.com/

• Kubernetes Documentation
https://kubernetes.io/docs/

• Terraform Documentation
https://developer.hashicorp.com/terraform/docs

• GitHub Documentation
https://docs.github.com/

• GitHub Actions Documentation
https://docs.github.com/en/actions

Github Repo Link : https://github.com/Nandinisharma-04Git/Multi-Agent_Insurance_System